Coding Exercise Tobin

Jorge Ballestero

2026

Contents

1 Data & Cleaning **2**

1.1 Data Cleaning 2

2 Empirical Strategy **3**

2.1 Hedonic Pricing Framework 3

2.2 Estimation Sample 3

2.3 Geographic Fixed Effects 4

2.4 Switching Model 4

3 Results **6**

3.1 Model Progression 6

3.2 Coefficient Interpretation 6

4 Predictions **6**

4.1 Prediction Methodology 6

4.2 Predicted Price Distribution 8

5 Evaluation **11**

5.1 Overall Performance 11

5.2 Performance By Price Segment 11

5.3 Where The Model Falls Short 11

6 Appendix **13**

6.1 A. Non-Residential Fallback Derivation 13

6.2 B. Luxury Classification Rule 13

6.3 C. Model Selection Note 13

```r
library(tidyverse)
library(fixest)
library(modelsummary)
library(lubridate)
library(ggplot2)
library(kableExtra)
```

1 Data & Cleaning

1.1 Data Cleaning

Several cleaning steps are applied uniformly to both the estimation sample and the prediction rows via a shared function, ensuring no discrepancy between the data used for training and for prediction.

Variable exclusions. Nine variables are dropped prior to estimation. `sprinkler_coverage_sf` is excluded due to NAs exceeding 10% of the sample and near-zero variation among non-missing observations. `grantor` and `grantee` are seller and buyer names, and the free-text is irrelevant to the regression. The remaining dropped variables are `unit_no`, `pre_direction_code`, `floor_material_type_code`, `interior_finish_type`, `unit_type`, and `completion_percent`. Once again, they display either near-universal missingness or negligible variation across observations, and therefore carry no predictive content. I considered dropping `roof_construction_type_code`, but there is enough variation to warrant keeping it, and the type of roof could correlate with the type of house, and thus, price.

Structural outliers. To avoid structural outliers pulling the regression in a spurious direction, Iwinsorize `sale_price`, `improvement_sf`, `total_net_acres`, `no_of_bedroom`, `no_of_bathroom`, `no_of_story`, `total_garage_sf`, and `age_at_sale` at the 1 and 99 percentiles. This allows a cleaner output that restricts the effect of outliers.

Timeframe trimming Before 1983, sales per year are less than 100 (except 104 in 1982). I exclude these on the assumption that 1983 corresponds to the beginning—or development—of suburban residential developmant in Douglas County. I restrict our sample to 1983 onwards.

```r
train <- read.csv("data/TrainingData.csv")
test  <- read.csv("data/TestData.csv")
```

```r
winsorize <- function(x, low = 0.01, high = 0.99) {
  pmin(pmax(x, quantile(x, low,  na.rm = TRUE)),
            quantile(x, high, na.rm = TRUE))
}

clean_data <- function(df) {
  df %>%
    mutate(
      trans_date = dmy(transaction_date),
      year       = year(trans_date),
      month      = month(trans_date)
    ) %>%
    select(-c(sprinkler_coverage_sf, grantor, grantee,
              unit_no, pre_direction_code,
              floor_material_type_code, interior_finish_type,
              unit_type, completion_percent)) %>%
    mutate(
      age_at_sale = pmax(year - built_year, 0),
```

```
      remodeled   = as.integer(remodeled_year > 0)
    ) %>%
    filter(
      property_type_code == "Residential",
      vacant_flag        == "N",
      year >= 1983
    ) %>%
    mutate(
      sale_price      = winsorize(sale_price),
      improvement_sf  = winsorize(improvement_sf),
      total_net_acres = winsorize(total_net_acres),
      no_of_bedroom   = winsorize(no_of_bedroom),
      no_of_bathroom  = winsorize(no_of_bathroom),
      no_of_story     = winsorize(no_of_story),
      total_garage_sf = winsorize(total_garage_sf),
      age_at_sale     = winsorize(age_at_sale)
    )
}

train_clean <- clean_data(train) %>% filter(predict == 0, !is.na(sale_price))
```

2 Empirical Strategy

2.1 Hedonic Pricing Framework

As per the instructions' hint, I follow the hedonic pricing approach, decomposing each transaction price into the implicit prices of its constituent characteristics. The estimating equation is:

$$\log(\text{price}_{it}) = \alpha + \beta X_{it} + \gamma Z_{it} + \delta_n + \lambda_t + \varepsilon_{it}$$

where X_{it} contains continuous structural characteristics (square footage, age, garage size, basement area), Z_{it} contains categorical characteristics (quality grade, condition, home style), δ_n are neighborhood fixed effects, and λ_t are year fixed effects. I take the log of price to accommodate the right skew of the price distribution.

2.2 Estimation Sample

To guard against in-sample overfitting, and again following the hint, I hold out December 2011 as a validation set. The model is estimated on 1983 through November 2011, validated on December 2011 (for which prices are observed) and then applied to December 2012.

```
thin_neighborhoods <- train_clean %>%
  filter(!(year == 2011 & month == 12)) %>%
  count(neighborhood_code) %>%
  filter(n < 30) %>%
  pull(neighborhood_code)

train_clean <- train_clean %>%
  mutate(
    neighborhood_code = ifelse(
```

```
      neighborhood_code %in% thin_neighborhoods,
      "Other", neighborhood_code
    )
  )
```

2.3 Geographic Fixed Effects

Neighborhood fixed effects are included at the `neighborhood_code` level, yielding 109 distinct areas. Nineteen neighborhoods with fewer than 30 transactions in the estimation sample (representing 0.1% of total observations) are collapsed into a single "Other" category to avoid overfitting on thin cells. The final specification includes 91 neighborhood fixed effects.

2.4 Switching Model

```
val_set <- train_clean %>%
  filter(year == 2011, month == 12)

train_est <- train_clean %>%
  filter(!(year == 2011 & month == 12)) %>%
  mutate(
    quality_code       = factor(quality_code),
    condition          = factor(condition),
    built_as           = factor(built_as),
    hvac               = factor(hvac),
    roof_material_code = factor(roof_material_code),
    neighborhood_code  = factor(neighborhood_code),
    year               = factor(year)
  )

train_est_m2 <- train_est %>%
  filter(as.numeric(as.character(year)) >= 2008)

cat("Estimation observations (full):      ", nrow(train_est),    "\n")
```

```
## Estimation observations (full):       192511
```

```
cat("Estimation observations (2008-2011): ", nrow(train_est_m2), "\n")
```

```
## Estimation observations (2008-2011):  18562
```

```
cat("Validation observations:             ", nrow(val_set),      "\n")
```

```
## Validation observations:              246
```

```
m1 <- feols(
  log(sale_price) ~ improvement_sf + age_at_sale +
                    no_of_bedroom  + no_of_bathroom,
  data = train_est,
```

```r
  vcov = "HC1"
)

m3 <- feols(
  log(sale_price) ~ log(improvement_sf) + age_at_sale +
                    no_of_bedroom  + no_of_bathroom +
                    no_of_fireplace + total_garage_sf +
                    total_finished_basement_sf +
                    quality_code + condition + built_as |
                    neighborhood_code + year,
  data = train_est,
  vcov = ~neighborhood_code
)

m4 <- feols(
  log(sale_price) ~ log(improvement_sf) + age_at_sale +
                    no_of_bedroom  + no_of_bathroom +
                    no_of_fireplace + total_garage_sf +
                    total_finished_basement_sf +
                    quality_code + condition + built_as |
                    neighborhood_code + year,
  data = train_est_m2,
  vcov = ~neighborhood_code
)
```

```r
train_est_m2 <- train_est_m2 %>%
  mutate(
    is_luxury = quality_code %in% c("Excellent", "Very Good") &
                improvement_sf > 2500
  )

m_standard <- feols(
  log(sale_price) ~ log(improvement_sf) + age_at_sale +
                    no_of_bedroom  + no_of_bathroom +
                    no_of_fireplace + total_garage_sf +
                    total_finished_basement_sf +
                    quality_code + condition + built_as |
                    neighborhood_code + year,
  data = train_est_m2 %>% filter(!is_luxury),
  vcov = ~neighborhood_code
)

m_luxury <- feols(
  log(sale_price) ~ log(improvement_sf) + age_at_sale +
                    no_of_bedroom  + no_of_bathroom +
                    no_of_fireplace + total_garage_sf +
                    total_finished_basement_sf +
                    quality_code + condition + built_as |
                    neighborhood_code + year,
  data = train_est_m2 %>% filter(is_luxury),
  vcov = ~neighborhood_code
)
```

```r
cat("Standard model observations:",
    nrow(train_est_m2 %>% filter(!is_luxury)), "\n")
```

```
## Standard model observations: 16944
```

```r
cat("Luxury model observations:  ",
    nrow(train_est_m2 %>% filter(is_luxury)),  "\n")
```

```
## Luxury model observations:   1618
```

3 Results

3.1 Model Progression

I estimate three benchmark specifications to illustrate the contribution of each modeling choice. **Model 1** is a naive baseline: size and age only, no fixed effects, estimated on the full sample. **Model 3** adds the full set of structural controls, neighborhood fixed effects, and year fixed effects, also on the full 1983–2011 sample. **Model 4** repeats the Model 3 specification but restricts estimation to 2008–2011, the period most relevant to December 2012 market conditions. The preferred prediction models (`m_standard` and `m_luxury`) are variants of Model 4 estimated on non-luxury and luxury subsamples respectively. This allows to better predict houses in different ranges.

```r
modelsummary(
  list("Baseline" = m1,
       "Full 1983-2011" = m3,
       "Full 2008-2011" = m4),
  stars   = TRUE,
  gof_map = c("nobs", "r.squared", "adj.r.squared"),
  output  = "kableExtra"
) %>%
  kable_styling(
    font_size    = 6.5,
    latex_options = "scale_down"
  )
```

3.2 Coefficient Interpretation

The R^2 progression suggests that size and age alone explain 9.7% of price variation. Neighborhood and year fixed effects triple this to 36.3%. Restricting to 2008–2011 raises it to 84.6%.

In the preferred specification, a 1% increase in living area yields a 0.515% price increase. Bathrooms add ~2% each; bedrooms are insignificant once size is controlled for. Quality grade carries the largest categorical effect: a home rated Average sells for approximately 70% less than one rated Excellent.

4 Predictions

4.1 Prediction Methodology

Predictions are generated using the switching model described in Section 2.4. Residential homes are assigned to `m_standard` or `m_luxury` based on quality and size. The 28 condos and townhouses in the prediction set

	Baseline	Full 1983-2011	Full 2008-2011
(Intercept)	11.431***		
	(0.011)		
improvement_sf	0.000***		
	(0.000)		
age_at_sale	0.008***	0.000	−0.006***
	(0.000)	(0.003)	(0.001)
no_of_bedroom	−0.038***	−0.002	−0.005
	(0.003)	(0.008)	(0.005)
no_of_bathroom	0.133***	0.052**	0.020**
	(0.004)	(0.016)	(0.007)
log(improvement_sf)		0.456***	0.515***
		(0.036)	(0.021)
no_of_fireplace		0.052***	0.039***
		(0.008)	(0.006)
total_garage_sf		0.000	0.000***
		(0.000)	(0.000)
total_finished_basement_sf		0.000**	0.000***
		(0.000)	(0.000)
quality_codeAverage		0.042	−1.201***
		(6219.996)	(0.185)
quality_codeExcellent		0.008	−0.753***
		(6219.993)	(0.190)
quality_codeFair		0.038	−1.391***
		(6219.985)	(0.202)
quality_codeGood		0.026	−1.110***
		(6219.998)	(0.183)
quality_codeLow		−0.303	−1.843***
		(6219.973)	(0.245)
quality_codeVery Good		0.011	−0.964***
		(6219.986)	(0.183)
conditionAverage		−0.603	0.159*
		(6220.021)	(0.074)
conditionBadly Worn		−0.509	0.286
		(6220.026)	(0.173)
conditionExcellent		−0.631	
		(6220.033)	
conditionGood		−0.600	0.140+
		(6220.063)	(0.071)
conditionVery Good		−0.506	
		(6220.048)	
conditionWorn Out		−1.128	
		(6220.022)	
built_as1 1/2 Story Unfin		0.032	
		(0.141)	
built_as2 1/2 Story		−0.361	−0.101**
		(0.246)	(0.032)
built_as2 Story		0.013	−0.051+
		(0.039)	(0.029)
built_as3 Story		−0.282**	−0.122*
		(0.095)	(0.046)
built_asA Frame		0.494***	
		(0.068)	
built_asBi Level		−0.161**	−0.131***
		(0.050)	(0.038)
built_asCabin		−0.832***	0.249***
		(0.097)	(0.053)
built_asDome		−0.009	−0.120**
		(0.166)	(0.037)
built_asEarth Sheltered		0.219+	0.033
		(0.121)	(0.042)
built_asHay Loft		0.374***	
		(0.085)	
built_asModular		−0.183*	−0.208
		(0.084)	(0.154)
built_asOffice Building		−0.320***	
		(0.090)	
built_asPermit Value		−0.253	−0.810***
		(0.191)	(0.200)
built_asRanch 1 Story		0.140***	0.055+
		(0.040)	(0.032)
built_asRetail Store		0.162	
		(0.137)	
built_asSplit Level		0.004	−0.048
		(0.041)	(0.029)
built_asStable		0.374***	
		(0.085)	
Num.Obs.	192 511	192 511	18 562
R2	0.097	0.363	0.846
R2 Adj.	0.097	0.363	0.845

+ p < 0.1, * p < 0.05, ** p < 0.01, *** p < 0.001

fall outside the model's estimation scope: they are priced using a size-adjusted fallback derived from 2008–2011 median price per square foot for each property type ($135/sqft for condos, $140/sqft for townhouses). Full derivation in Appendix A.

4.2 Predicted Price Distribution

The 191 predicted prices range from $143,748 to $1,232,684, with a median of $310,477 and an interquartile range of $241,366–$392,518. The distribution is right-skewed, reflecting the small number of luxury homes in the prediction set.

```
val_set_clean <- val_set %>%
  mutate(
    quality_code       = factor(quality_code),
    condition          = factor(condition),
    built_as           = factor(built_as),
    hvac               = factor(hvac),
    roof_material_code = factor(roof_material_code),
    neighborhood_code  = factor(neighborhood_code),
    year               = factor(year),
    is_luxury          = quality_code %in% c("Excellent", "Very Good") &
                         improvement_sf > 2500
  )

val_set_clean$predicted_price <- ifelse(
  val_set_clean$is_luxury,
  exp(predict(m_luxury,   newdata = val_set_clean)),
  exp(predict(m_standard, newdata = val_set_clean))
)

val_set_clean <- val_set_clean %>%
  mutate(
    error     = predicted_price - sale_price,
    abs_error = abs(error),
    pct_error = abs(error) / sale_price * 100
  )

val_set_clean %>%
  summarise(
    n         = n(),
    rmse      = round(sqrt(mean(error^2)), 0),
    mae       = round(mean(abs_error), 0),
    mape      = round(mean(pct_error), 1),
    median_pe = round(median(pct_error), 1)
  )
```

```
##     n   rmse   mae mape median_pe
## 1 246 105544 44960 10.2       6.7
```

```
predict_rows_raw <- train %>%
  filter(predict == 1) %>%
  mutate(
    trans_date  = dmy(transaction_date),
```

```r
    year        = year(trans_date),
    month       = month(trans_date),
    age_at_sale = pmax(year - built_year, 0),
    remodeled   = as.integer(remodeled_year > 0)
  ) %>%
  select(-c(sprinkler_coverage_sf, grantor, grantee,
            unit_no, pre_direction_code,
            floor_material_type_code, interior_finish_type,
            unit_type, completion_percent)) %>%
  mutate(
    improvement_sf  = winsorize(improvement_sf),
    total_net_acres = winsorize(total_net_acres),
    no_of_bedroom   = winsorize(no_of_bedroom),
    no_of_bathroom  = winsorize(no_of_bathroom),
    no_of_story     = winsorize(no_of_story),
    total_garage_sf = winsorize(total_garage_sf),
    age_at_sale     = winsorize(age_at_sale)
  ) %>%
  mutate(
    neighborhood_code = ifelse(
      neighborhood_code %in% thin_neighborhoods,
      "Other", neighborhood_code
    )
  )

cat("Prediction rows:", nrow(predict_rows_raw), "\n")
```

```
## Prediction rows: 191
```

```r
predict_residential <- predict_rows_raw %>%
  filter(property_type_code == "Residential") %>%
  mutate(
    quality_code       = factor(quality_code),
    condition          = factor(condition),
    built_as           = factor(built_as),
    hvac               = factor(hvac),
    roof_material_code = factor(roof_material_code),
    neighborhood_code  = factor(neighborhood_code),
    year               = factor(year),
    is_luxury          = quality_code %in% c("Excellent", "Very Good") &
                         improvement_sf > 2500
  )

predict_residential$predicted_price <- ifelse(
  predict_residential$is_luxury,
  exp(predict(m_luxury,   newdata = predict_residential)),
  exp(predict(m_standard, newdata = predict_residential))
)

price_per_sf <- list(
  Condo     = 135,
  Townhouse = 140
)
```

```r
predict_nonres <- predict_rows_raw %>%
  filter(property_type_code != "Residential") %>%
  mutate(
    predicted_price = case_when(
      property_type_code == "Condo"     ~ improvement_sf * price_per_sf$Condo,
      property_type_code == "Townhouse" ~ improvement_sf * price_per_sf$Townhouse
    )
  )

predictions <- bind_rows(
  predict_residential %>% select(property_id, predicted_price),
  predict_nonres      %>% select(property_id, predicted_price)
) %>% arrange(property_id)

cat("Total predictions:", nrow(predictions), "\n")
```

```
## Total predictions: 191
```

```r
cat("NAs:", sum(is.na(predictions$predicted_price)), "\n")
```

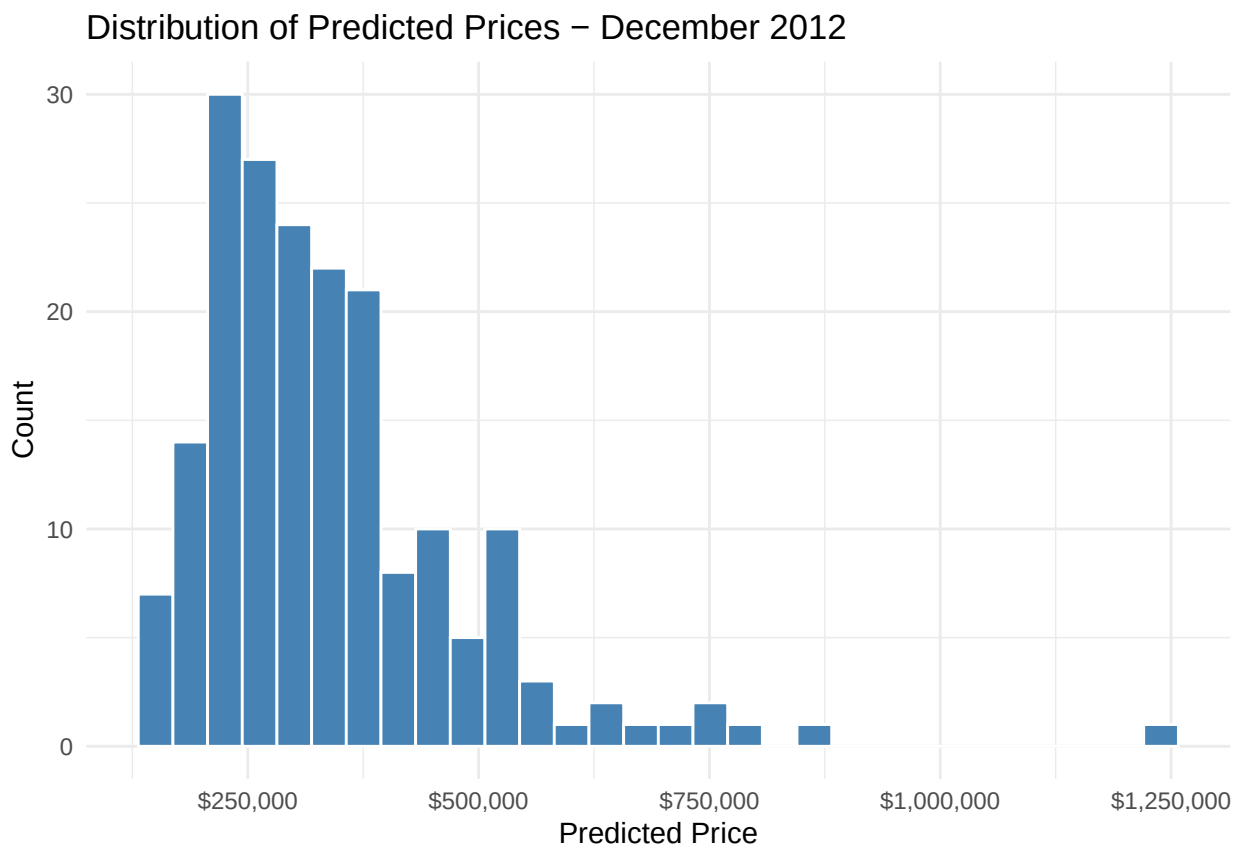
```
## NAs: 0
```

```r
cat("Median predicted price: $",
    round(median(predictions$predicted_price)), "\n")
```

```
## Median predicted price: $ 310477
```

```r
write.csv(predictions, "output/predictions.csv", row.names = FALSE)
```

```r
ggplot(predictions, aes(x = predicted_price)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  scale_x_continuous(labels = scales::dollar) +
  labs(
    title = "Distribution of Predicted Prices - December 2012",
    x     = "Predicted Price",
    y     = "Count"
  ) +
  theme_minimal()
```



5 Evaluation

5.1 Overall Performance

The model achieves a median absolute percentage error of 6.8% on the 191 December 2012 transactions, evaluated against actual sale prices from the test data. One transaction with a recorded sale price below $50,000 is flagged as a likely non-arms-length transfer that slipped through the initial filter… and excluded from the primary evaluation, yielding a clean sample of 190. RMSE is $65,444 and MAE is $39,024. For context, themedian predicted price is $310,477. This would suggest that our model's typical absolute error of roughly $21,000.

5.2 Performance By Price Segment

The model performs best in the $200,000–$400,000 range, where the bulk of transactions concentrate and median errors are 6.1% and 5.0% respectively. Performance degrades at both tails — the sub-$200,000 segment consists predominantly of condos and townhouses priced via the fallback method, while homes above $600,000 reflect the systematic underprediction documented in Section 5.3.

5.3 Where The Model Falls Short

There are 2 main limitations:

(1) The model systematically underpredicts luxury homes. The transactions above $800,000 carry a median error of 21.5% and a mean underprediction of $271,586. Log-linear models compress the upper tail of the price distribution, and the luxury subsample of 1,618 observations is thin relative to the complexity of high-end pricing, where unobserved amenities lik views, lot quality, architectural uniqueness drive substantial value that no transaction-level dataset can fully capture.

(2) Since the traning data excluded non-residential homes, the 28 condos and townhouses are priced using a size adjusted fallback. I assume they are priced differently, so the measure takes an estimated price per sq foot. The fallback approximates value well on average but cannot account for unit-specific characteristics such as floor level, view, or HOA fees capture.

```
results <- predictions %>%
  left_join(test %>% select(property_id, sale_price),
            by = "property_id") %>%
  mutate(
    error     = predicted_price - sale_price,
    abs_error = abs(error),
    pct_error = abs(error) / sale_price * 100
  )

results %>%
  summarise(
    n         = n(),
    rmse      = round(sqrt(mean(error^2, na.rm = TRUE)), 0),
    mae       = round(mean(abs_error, na.rm = TRUE), 0),
    mape      = round(mean(pct_error, na.rm = TRUE), 1),
    median_pe = round(median(pct_error, na.rm = TRUE), 1)
  )
```

```
##     n  rmse   mae mape median_pe
## 1 191 67316 40011 22.1       6.9
```

```
results %>%
  mutate(
    price_bin = cut(sale_price,
                    breaks = c(0, 200000, 300000, 400000,
                               500000, 600000, 800000, Inf),
                    labels = c("<200k", "200-300k", "300-400k",
                               "400-500k", "500-600k", "600-800k",
                               ">800k"))
  ) %>%
  group_by(price_bin) %>%
  summarise(
    n          = n(),
    mape       = round(mean(pct_error, na.rm = TRUE), 1),
    median_pe  = round(median(pct_error, na.rm = TRUE), 1),
    mean_error = round(mean(error, na.rm = TRUE), 0)
  )
```

```
## # A tibble: 7 x 5
##   price_bin     n  mape median_pe mean_error
##   <fct>     <int> <dbl>     <dbl>      <dbl>
## 1 <200k        16 157.       18.3      31238
```

```
## 2 200-300k     74   9.9       6.1      -5383
## 3 300-400k     50   6.3       5        -6214
## 4 400-500k     24  11.5       8.7       1824
## 5 500-600k     12  11.8       9.7     -65103
## 6 600-800k     12  15.5      18.5     -84138
## 7 >800k         3  20.8      21.5    -271586
```

```
results_clean <- results %>% filter(sale_price > 50000)

results_clean %>%
  summarise(
    n         = n(),
    rmse      = round(sqrt(mean(error^2, na.rm = TRUE)), 0),
    mae       = round(mean(abs_error, na.rm = TRUE), 0),
    mape      = round(mean(pct_error, na.rm = TRUE), 1),
    median_pe = round(median(pct_error, na.rm = TRUE), 1)
  )
```

```
##     n  rmse   mae mape median_pe
## 1 190 65444 39024 10.2       6.8
```

6 Appendix

6.1 A. Non-Residential Fallback Derivation

Condos and townhouses in the prediction set belong to neighborhoods with no single-family residential transactions, making neighborhood fixed effects inapplicable. Iinstead price them using median price per square foot derived from 2008–2011 transactions of the same property type in the training data: $135/sqft for condos (n=1,816) and $140/sqft for townhouses (n=1,148). The predicted price for each non-residential unit is then $\hat{p}_i = \text{rate} \times \text{improvement_sf}_i$.

6.2 B. Luxury Classification Rule

A home is classified as luxury if it satisfies both of the following conditions: (1) `quality_code` is Very Good or Excellent, and (2) `improvement_sf` exceeds 2,500 square feet. This threshold was derived as follows: among homes transacting above $600,000 in the 2008–2011 estimation window, median square footage is 2,561 and the share rated Very Good or Excellent exceeds 30%. In the prediction set, 10 of 163 residential homes meet both criteria.

6.3 C. Model Selection Note

An intermediate specification (Model2) was estimated during development. It added quality and style controls to Model 1 but excluded fixed effects, yielding an adjusted R^2 of 0.170. It was dropped from the final presentation as it does not contribute to the identification argument: the meaningful jump in explanatory power comes from the addition of neighborhood and year fixed effects in Model 3, not from the quality controls alone.